

IPL MATCH PREDICTOR

Project Report

Java Swing Desktop Application

Student Name: _____

Roll Number: _____

Course / Branch: _____

College: _____

Academic Year: _____

1. Abstract

IPL Match Predictor is a Java-based desktop application developed using Java Swing. The application provides a graphical interface through which a user selects two Indian Premier League (IPL) franchises and obtains a projected winner along with normalized percentage values for the two teams.

The current implementation uses a simple rule-based scoring method rather than a machine-learning model. Each supported team is assigned a fixed strength score in the program. For a selected matchup, the two strength values are converted into percentages by dividing each team's strength by the sum of both strengths. The application then identifies the team with the higher score as the predicted winner.

The project demonstrates GUI development, event-driven programming, data structures, image handling, validation, and basic numerical computation in Java.

2. Introduction

Sports prediction applications commonly combine historical data, statistics, and predictive models to estimate possible outcomes. This project presents a simplified IPL prediction application intended to demonstrate the basic workflow of a prediction interface without requiring an external database or machine-learning library.

The application focuses on usability. Users can select two teams, view their logos when available, click a prediction button, and immediately see the calculated result. The graphical interface is implemented with Java Swing and uses a dark theme with an accent color for the prediction output.

3. Problem Statement

A user-friendly interface is required to compare two IPL teams and provide an easily understandable projected outcome. Manually comparing team strength can be inconvenient, so the application automates the comparison and presents the result in a desktop graphical interface.

4. Objectives

- Develop a desktop GUI for IPL team comparison.
- Allow users to select two different IPL franchises.
- Associate each supported team with a strength score.
- Calculate normalized percentages for the selected teams.
- Display the team with the higher strength as the projected winner.
- Display team logos when image assets are available.
- Handle missing logos through a text-based abbreviation fallback.
- Demonstrate event-driven programming using Java Swing listeners.

5. Scope

The current project covers eight IPL franchises represented in the source code. The system is designed as a lightweight desktop demonstration and does not currently retrieve live IPL data or train a predictive model.

6. Technologies Used

Programming Language: Java

GUI Framework: Java Swing

Supporting APIs: java.awt, java.io.File, java.util.LinkedHashMap, and java.util.Map

7. System Design

The application can be viewed as four main layers: user GUI, event handling, prediction logic, and result display. The user selects two teams through Swing combo boxes. Event listeners update team visuals and invoke the prediction routine. The prediction routine reads the fixed strength values, calculates normalized percentages, determines the higher-strength team, and updates the result card.

8. Data Representation

Two LinkedHashMap objects are used. The teamLogos map stores team names and relative PNG paths, while the teamStrength map stores team names and integer strength values. LinkedHashMap preserves insertion order, which also determines the order of the teams in the dropdown menus.

9. User Interface

The main window contains three primary areas: a header, prediction controls, and a result card. The header shows the application title and subtitle. The prediction controls provide Team 1 and Team 2 dropdowns, a VS label, team logo/abbreviation areas, and the Predict Winner button. The result card displays the projected winner and normalized percentage values. The application uses custom colors and fonts to create a dark, modern visual style.

10. Prediction Algorithm

Let S1 be Team 1 strength and S2 be Team 2 strength.

$$\text{Total} = S1 + S2$$

$$P1 = (S1 / \text{Total}) \times 100$$

$$P2 = (S2 / \text{Total}) \times 100$$

The source code rounds each percentage to one decimal place. The winner is determined by the larger strength value. If the strengths are equal, Team 1 is selected because the implementation uses $t1 \geq t2$.

11. Example Calculation

For Mumbai Indians (90) versus Chennai Super Kings (99):

$$\text{Total} = 90 + 99 = 189$$

$$\text{Mumbai Indians} = 90 / 189 \times 100 = 47.6\%$$

$$\text{Chennai Super Kings} = 99 / 189 \times 100 = 52.4\%$$

The interface therefore displays Chennai Super Kings as the projected winner for this configuration.

12. Image Handling

The application attempts to load a team PNG logo using a relative filesystem path. The loadLogo() method checks whether the file exists before loading it. If the logo is unavailable, styleBadge() removes the icon and displays an abbreviation generated from the team name. This fallback means missing image assets do not stop the core application from running.

13. Input Validation

The application checks whether the same team has been selected in both dropdown menus. If both selections are identical, a warning dialog is shown: "Select two different teams!" The prediction is not calculated until two different teams are selected.

14. Program Execution Flow

Start → Create IPLPredictor object → Load team data → Build Swing interface → User selects two teams → User clicks Predict Winner → Validate selection → Read strength values → Calculate percentages → Determine higher strength → Update result card.

15. Testing

Test Case 1 - Valid team selection: Mumbai Indians vs Chennai Super Kings. Expected: percentages and a projected winner are displayed. Result: Pass.

Test Case 2 - Same team selected twice. Expected: warning dialog and no prediction update. Result: Pass.

Test Case 3 - Missing logo. Expected: team abbreviation instead of image. Result: Pass according to the implemented fallback logic.

Test Case 4 - Application startup. Expected: Swing window opens after successful compilation and execution. Result: Pass.

16. Limitations

The current implementation should be described as a rule-based demonstration, not as a machine-learning IPL prediction model. Team strengths are manually assigned, the score does not update from recent matches, no historical dataset is used, player statistics and venue conditions are ignored, and the displayed percentages are normalized strength shares rather than calibrated real-world probabilities.

17. Future Enhancements

1. Collect historical IPL match data.
2. Add team, player, venue, toss, and recent-form features.
3. Train and validate a machine-learning model.
4. Evaluate using appropriate metrics such as accuracy and log loss.
5. Persist data using CSV, SQLite, MySQL, or another database.
6. Add performance charts and historical head-to-head views.
7. Add prediction history and model evaluation screens.
8. Package the application as a Windows executable.

18. Advantages

- Simple and easy to understand.
- No third-party machine-learning libraries are required.
- Fast calculations.
- Lightweight desktop application.
- User-friendly graphical interface.
- Graceful handling of missing logo images.
- Clear separation between data setup, UI construction, and prediction logic.

19. Conclusion

IPL Match Predictor demonstrates how Java and Swing can be used to create an interactive sports comparison application. The program stores team information, accepts user selections, performs a deterministic strength-based calculation, and presents the result in a graphical interface.

The project provides a practical foundation for a more advanced IPL analytics system. Its most significant next step would be replacing fixed, manually assigned team strengths with real historical data and a validated statistical or machine-learning model.

Team Strength Table

Team	Strength
Mumbai Indians	90
Chennai Super Kings	99
Royal Challengers Bengaluru	70
Kolkata Knight Riders	75
Rajasthan Royals	70
Punjab Kings	65
Delhi Capitals	60
Lucknow Super Giants	55